COLLEGE PROJECT REPORT

LegalEase

Template-BasedLegal DocumentGenerator withAI-Ready Architecture

Submitted By	V.S.Sahana S.Rithika Sri
Register No.	212503172 212503170
Course	Bachelor of Computer Applications (BCA)
Department	Computer Applications
Academic Year	2026–2027
Project Type	Web Technology / AI Application
Technology	Python, Flask, ReportLab, HTML5, CSS3

Submitted in partial fulfillment of the requirements for the BCA programme.

CERTIFICATE

Thisi stocertifythattheproject entitled **“LegalEase: Template-Based Legal Document Generator with AI-Ready Architecture”** is a bonafide academic project carried out as part of the Bachelor of Computer Applications programme. The work demonstrates the design, implementation and testing of a web-based prototype for structured legal-document drafting.

Project Guide	Head of Department
_____	_____
Date: _____	Date: _____

DECLARATION

I herebyd eclare thatthis report is prepared for academic purposes and describes a prototype implementation of LegalEase. The generated legal text is illustrative and must not be treated as legal advice or as a substitute for review by a qualified legal professional.

ACKNOWLEDGEMENT

Ie xpressmysincere gratitude tothe faculty and the department for their guidance and support. I also thank my classmates and family for their encouragement during the design, coding, testing and documentation stages.

TABLE OF CONTENTS

- Abstract
- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Scope
- 5. Existing System
- 6. Proposed System
- 7. System Requirements
- 8. System Analysis
- 9. System Architecture
- 10. Modules
- 11. Data Flow Diagrams
- 12. Data Model
- 13. Algorithm
- 14. Complete Source Code (app, validators, clauses, PDF, templates, tests, AI layer)
- 15. Testing and Results
- 16. Output Screens
- 17. Advantages and Limitations
- 18. Future Enhancements
- 19. Viva Questions and Answers
- 20. Conclusion
- 21. Bibliography

ABSTRACT

LegalEase isa web-based academic prototype that shows how structured legal-document drafts can be generated from a small set of user inputs. The user chooses a document type (rental, NDA, employment, or leave-and-license) and enters party names, purpose, duration, amount, deposit, notice period and place. The server validates and sanitises the input, fills a predefined clause template, shows a preview and produces a downloadable PDF with signature and witness blocks.

The current version uses **deterministic templates** so it runs offline without any API key. The architecture is **AI-ready**: an optional module (ai_layer.py) can rewrite clause wording using a language model while the template remains the fallback. All output is presented as an educational draft that requires professional legal review.

1. INTRODUCTION

Legald ocuments contain repeated structures, formal terminology and many sections that beginners find hard to organise. LegalEase demonstrates a simplified digital workflow: structured input becomes a formatted document draft. The project combines a web interface, server-side validation, clause generation and PDF creation, split into small modules (validators, clauses, PDF utilities, routes) so each part can be tested and replaced independently.

2. PROBLEM STATEMENT

Students and small organisationsstru ggle to produce consistently formatted first drafts of common agreements. Manual preparation involves repetitive typing, formatting and clause organisation, and mistakes such as missing clauses or inconsistent names are common. LegalEase addresses this academic problem with a guided form and automatic draft generation. It does not decide legal rights, give personalised legal advice or replace professional review.

3. OBJECTIVES

- Providea simple,mobile-friendly web form for entering document details.
- Generate structured drafts (7–8 clauses) for four common agreement types.
- Validate and sanitise all input on the server; reject bad values with clear messages.
- Produce a professional PDF with signature and witness blocks.
- Demonstrate modular architecture with automated unit tests.
- Keep a clear legal/educational disclaimer in the UI and in every PDF.
- Provide an optional, safely-fenced AI layer for future clause refinement.

4. SCOPE

Inscope: rental,NDA, employment and leave-and-license agreements; form validation; preview; PDF download; unit tests. **Out of scope:** legal research, jurisdiction-specific validation, case-law analysis, electronic signatures and storage of user documents. Future versions may add authentication, history, multilingual support and legally reviewed template libraries.

5. EXISTING SYSTEM

Peopleu suallydraft agreementsinword processors using copied templates, or use online form-based document services. These approaches need manual searching for templates, copying data between fields,

and manual formatting, and customisation differs between services. This report treats the existing approach only as a reference point, not as a claim about every legal-document workflow.

Aspect	Manual / Word template	LegalEase prototype
Data entry	Edit text by hand in many places	One guided form
Consistency	Names/amounts may mismatch	Same value inserted everywhere
Validation	None	Server-side checks
Output	Manual formatting	Auto-formatted PDF with signature block
Speed	Slow for repeated documents	Seconds

6. PROPOSED SYSTEM

LegalEase collects only the fields the selected template needs. The Flask server validates the input, inserts it into a predefined clause structure, renders a preview and creates the PDF from the same validated data, so preview and PDF cannot drift apart. User text is XML-escaped before it reaches ReportLab, which prevents broken PDFs and markup injection. An AI model can later be placed between validation and clause generation, always with a template fallback and human review.

7. SYSTEM REQUIREMENTS

Category	Requirement
Hardware	Laptop/desktop, 4 GB RAM or more, modern processor
Operating system	Windows, Linux or macOS
Software	Python 3.9+, Flask 3.x, ReportLab 4.x
Front end	HTML5, CSS3, any modern browser
Optional	anthropic package + API key for AI layer; internet only for that layer

How to run

```
pipinstall flask reportlab
python app.py
                                  # open http://127.0.0.1:5000
python -m unittest -v tests.test_app    # run the 10 tests
FLASK_DEBUG=1python app.py        # debug mode (development only)
```

8. SYSTEM ANALYSIS

Stage	Details
Inputs	document_type, name, other_party, purpose, duration, amount, deposit, notice, place
Processing	Validate → sanitise → select template → build clauses → render preview → build PDF
Outputs	HTML preview page; downloadable PDF (A4) with disclaimer, date and signature blocks
Errors	HTTP 400 with readable messages for invalid type, amount, or identical parties

9. SYSTEM ARCHITECTURE

The browser sends form data to Flask routes. validators.py cleans and checks it; clauses.py returns the title and clause list; the result page shows the preview; pdf_utils.py creates the PDF in memory. Presentation (templates), business logic (clauses, validators) and output (PDF) are kept separate.

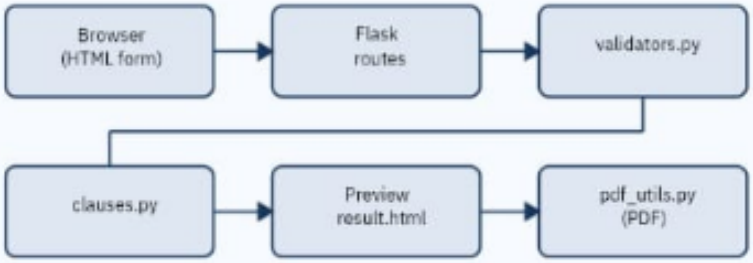


Figure: Architecture flow

10. MODULES

#	Module	File	Responsibility
1	User Interface	templates/index.html	Collect inputs, show validation errors
2	Routing	app.py	Handle /, /generate, /download
3	Validation	validators.py	Defaults, amount check, length limit, escaping
4	Draft Generation	clauses.py	Clause library for four agreement types
5	Preview	templates/result.html	Display clauses; hidden fields for download
6	PDF	pdf_utils.py	Build PDF with signature block and disclaimer
7	AI Layer (optional)	ai_layer.py	Refine wording with fallback to template
8	Testing	tests/test_app.py	10 automated test cases

11. DATA FLOW DIAGRAMS

Level 0 (context diagram)



Figure: DFD Level 0 – user gives input; system returns preview and PDF

Level 1



Figure: DFD Level 1 – internal processes (no database data store in the prototype)

12. DATA MODEL

The prototypes toresnothing: form fields are temporary request data, and the PDF is generated in memory. For an expanded version, a relational schema is proposed below. Sensitive data should be minimised and encrypted.

Table

User	Columns
Template	user_id (PK), name, email (unique), password_hash, created_at
Document	template_id (PK), doc_type, version, clause_json, reviewed_by_lawyer (bool)
AuditLog	document_id (PK), user_id (FK), template_id (FK), created_at, status, file_path
	log_id (PK), user_id (FK), action, timestamp, ip_address

13. ALGORITHM

- **Step 1:** Startth e Flask application and display the form.
- **Step 2:** Receive POST data with document type and details.
- **Step 3:** Validate: type is allowed; amounts match \d{1,9}(.\d{1,2})?; parties differ; text limited to 200 characters; blank fields get defaults.
- **Step 4:** If errors exist, re-display the form with messages (HTTP 400).
- **Step 5:** Select the clause builder for the chosen type and generate clauses.
- **Step 6:** Display the preview page with a Download PDF button.
- **Step 7:** On download, re-validate, escape text, build the PDF in memory and return it.
- **Step 8:** End.

14. COMPLETE SOURCE CODE

The project folder structure is shownf irst,followedby each file. All code below was executed and passed the 10 unit tests listed in Section 15.

```
legalease/
 |-- app.py
 |-- validators.py
 |-- clauses.py
 |-- pdf_utils.py          #Flask routes (entry point)
 |-- ai_layer.py           #validation, defaults, escaping
 |-- templates/            #clause libraries for 4 agreement types
                           #ReportLab PDF builder
                           #optional AI clause refinement

  |    |-- index.html     #input form
|      `-- result.html    #preview page
 `-- tests/
       `-- test_app.py    #unit tests
```

14.1 app.py

```
"""LegalEase - Flask application (main entry point)."""
import os
from flask import Flask, render_template, request, send_file
from clauses import generate_document
from pdf_utils import build_pdf
from validators import validate


app = Flask(__name__)

DOCUMENT_TYPES = {
    "rental": "Rental Agreement",
    "nda": "Non-Disclosure Agreement",
    "employment": "Employment Agreement",
    "leave": "Leave and License Agreement",
}


@app.route("/")
def index():
    return render_template("index.html", document_types=DOCUMENT_TYPES, errors=[], form={})


@app.route("/generate", methods=["POST"])
def generate():
    data, errors = validate(request.form)
    if errors:
        return render_template("index.html", document_types=DOCUMENT_TYPES,
                               errors=errors, form=request.form), 400
    title, clauses = generate_document(data["document_type"], data)
    return render_template("result.html", title=title, clauses=clauses, data=data)


@app.route("/download", methods=["POST"])
def download():
    data, errors = validate(request.form)
    if errors:
        return {"errors": errors}, 400
    title, clauses = generate_document(data["document_type"], data)
    pdf = build_pdf(title, clauses, data)
    return send_file(pdf, as_attachment=True, download_name="LegalEase_Document.pdf",
                     mimetype="application/pdf")


if __name__ == "__main__":
    app.run(debug=os.environ.get("FLASK_DEBUG") == "1")
```

14.2 validators.py

```
"""Input validation and sanitisation for LegalEase."""
import re
from xml.sax.saxutils import escape

DEFAULTS = {
    "name": "Sample User",
    "other_party": "Sample Party",
    "purpose": "General legal arrangement",
    "duration": "12 months",
    "amount": "10000",
    "deposit": "0",
    "notice": "30 days",
    "place": "India",
}
MAX_LEN = 200
VALID_TYPES = {"rental", "nda", "employment", "leave"}


def clean_text(value, default):
    """Strip whitespace, apply default for blank values, limit length."""
    value = (value or "").strip()
    return (value or default)[:MAX_LEN]


def clean_amount(value, default):
    """Return amount as digits with thousand separators, or raise ValueError."""
    raw = (value or "").replace(",", "").strip() or default
    if not re.fullmatch(r"\d{1,9}(\.\d{1,2})?", raw):
        raise ValueError("Amount must be a positive number (max 9 digits).")
    num = float(raw)
    return f"{num:,.0f}" if num.is_integer() else f"{num:,.2f}"


def validate(form):
    """Validate a form dict. Returns (data, errors)."""
    errors, data = [], {}
    doc_type = form.get("document_type", "rental")
    if doc_type not in VALID_TYPES:
        errors.append("Unknown document type.")
    data["document_type"] = doc_type
    for key in ("name", "other_party", "purpose", "duration", "notice", "place"):
        data[key] = clean_text(form.get(key), DEFAULTS[key])
    for key in ("amount", "deposit"):
        try:
            data[key] = clean_amount(form.get(key), DEFAULTS[key])
        except ValueError as exc:
            errors.append(f"{key.title()}: {exc}")
            data[key] = DEFAULTS[key]
    if data["name"].lower() == data["other_party"].lower():
        errors.append("The two parties must have different names.")
    return data, errors


def safe(text):
    """Escape text for ReportLab Paragraph markup."""
    return escape(text)
```

14.3 clauses.py

```
"""Clause libraries: each builder returns (title, [(heading, text), ...])."""


def rental(d):
    return "RENTAL AGREEMENT", [
        ("Parties", f"This agreement is made at {d['place']} between {d['name']} (the Tenant) and {d['other_party']}
        (the Landlord)."),
        ("Premises", f"The Landlord lets the premises described as: {d['purpose']}."),
        ("Term", f"The tenancy is for {d['duration']} from the date of signing."),
        ("Rent", f"The Tenant shall pay monthly rent of Rs. {d['amount']} on or before the 5th of every month."),
        ("Security Deposit", f"The Tenant has paid an interest-free refundable deposit of Rs. {d['deposit']},
        returnable on vacating after adjusting lawful dues."),
        ("Maintenance", "The Tenant shall keep the premises in good condition. Structural repairs are the Landlord's
        responsibility."),
        ("Termination", f"Either party may terminate this agreement by giving {d['notice']} written notice."),
        ("Legal Compliance", "The parties shall comply with applicable law, including local stamp-duty and
        registration requirements."),

    ]


def nda(d):
    return"NON-DISCLOSURE AGREEMENT", [
        ("Parties", f"This agreement is made at {d['place']} between {d['name']} (the Receiving Party) and
        {d['other_party']} (the Disclosing Party)."),
        ("Confidential Information", f"Confidential Information means all non-public information relating to:
        {d['purpose']}."),
        ("Obligations", "The Receiving Party shall use the information only for the stated purpose and protect it with
         reasonable care."),
        ("Exclusions", "Information that is public, already known, or independently developed is not confidential."),
        ("Term", f"The confidentiality obligations continue for {d['duration']}."),
        ("Remedies", "The parties acknowledge that breach may cause harm and that the Disclosing Party may seek
        appropriate legal remedies."),
        ("Return of Material", f"On request, or within {d['notice']} of termination, all confidential material shall
        bereturned or destroyed."),

    ]


def employment(d):
    return"EMPLOYMENT AGREEMENT", [
        ("Parties", f"This agreement is made at {d['place']} between {d['name']} (the Employee) and {d['other_party']}
         (the Employer)."),
        ("Position", f"The Employee is appointed for the role of: {d['purpose']}."),
        ("Term", f"The employment period is {d['duration']}, subject to a probation review."),
        ("Remuneration", f"The Employer shall pay a monthly salary of Rs. {d['amount']}, subject to statutory
        deductions."),
        ("Duties", "The Employee shall perform assigned duties diligently and follow company policies."),
        ("Confidentiality", "The Employee shall not disclose confidential business information during or after
        employment."),
        ("Termination", f"Either party may end this agreement with {d['notice']} written notice."),

    ]


def leave(d):
    return"LEAVE AND LICENSE AGREEMENT", [
        ("Parties", f"This agreement is made at {d['place']} between {d['name']} (the Licensee) and {d['other_party']}
         (the Licensor)."),
        ("Licensed Premises", f"The Licensor grants a licence to use: {d['purpose']}."),
        ("Licence Period", f"The licence period is {d['duration']}."),
        ("Licence Fee", f"The Licensee shall pay a monthly licence fee of Rs. {d['amount']}."),
        ("Security Deposit", f"A refundable deposit of Rs. {d['deposit']} has been paid to the Licensor."),
        ("Nature of Licence", "This is a personal licence and does not create any tenancy or interest in the
        property."),
        ("Termination", f"Either party may terminate this licence with {d['notice']} written notice."),
        ("Registration", "The parties shall complete registration and stamp-duty formalities as required by local
        law."),

    ]


BUILDERS = {"rental": rental, "nda": nda, "employment": employment, "leave": leave}


def generate_document(doc_type, data):
    return BUILDERS[doc_type](data)
```

14.8 tests/test_app.py

```
import os
import sys
import unittest

sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
from app import app
from validators import clean_amount, validate

GOOD = {"document_type": "rental", "name": "Janani R", "other_party": "Ravi K",
        "purpose": "2BHK flat", "duration": "11 months", "amount": "15000"}


class LegalEaseTests(unittest.TestCase):
    def setUp(self):
        app.config["TESTING"] = True
        self.client = app.test_client()

    def test_01_home_page(self):
        self.assertEqual(self.client.get("/").status_code, 200)

    def test_02_valid_rental_preview(self):
        r = self.client.post("/generate", data=GOOD)
        self.assertEqual(r.status_code, 200)
        self.assertIn(b"RENTAL AGREEMENT", r.data)

    def test_03_empty_fields_use_defaults(self):
        data, errors = validate({"document_type": "nda"})
        self.assertFalse(errors)
        self.assertEqual(data["name"], "Sample User")

    def test_04_invalid_amount_rejected(self):
        r = self.client.post("/generate", data={**GOOD, "amount": "abc"})
        self.assertEqual(r.status_code, 400)

    def test_05_negative_amount_rejected(self):
        with self.assertRaises(ValueError):
            clean_amount("-500", "0")

    def test_06_same_party_rejected(self):
        _, errors = validate({**GOOD, "other_party": "janani r"})
        self.assertTrue(errors)

    def test_07_unknown_type_rejected(self):
        r = self.client.post("/generate", data={**GOOD, "document_type": "x"})
        self.assertEqual(r.status_code, 400)

    def test_08_all_types_pdf(self):
        for t in ("rental", "nda", "employment", "leave"):
            r = self.client.post("/download", data={**GOOD, "document_type": t})
            self.assertEqual(r.status_code, 200, t)
            self.assertTrue(r.data.startswith(b"%PDF"), t)

    def test_09_special_characters_pdf(self):
        r = self.client.post("/download", data={**GOOD, "name": "A & B <Ltd>", "purpose": "<b>x"})
        self.assertEqual(r.status_code, 200)
        self.assertTrue(r.data.startswith(b"%PDF"))

    def test_10_long_text_truncated(self):
        data, _ = validate({**GOOD, "purpose": "x" * 1000})
        self.assertEqual(len(data["purpose"]), 200)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Note: ai_layer.py needs network access and an API key, so it is not exercised by the tests; without a key it returns the template text unchanged. Any AI-produced clause must be reviewed by a qualified professional.

14.4 pdf_utils.py

```
"""PDF creation with ReportLab."""
from datetime import date
from io import BytesIO
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import mm
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle
from validators import safe


NAVY = colors.HexColor("#17365D")
DISCLAIMER = ("<b>Educational prototype disclaimer:</b> This text is for academic "
              "demonstration only and is not legal advice. A qualified legal "
              "professional should review any document before real-world use.")


def build_pdf(title, clauses, data):
    styles = getSampleStyleSheet()
    body = ParagraphStyle("body", parent=styles["BodyText"], leading=15)
    buf = BytesIO()
    doc = SimpleDocTemplate(buf, pagesize=A4, leftMargin=18*mm, rightMargin=18*mm,
                            topMargin=18*mm, bottomMargin=18*mm, title=title)
    story = [
        Paragraph("LegalEase", ParagraphStyle("brand", parent=styles["Title"],
                  alignment=TA_CENTER, fontSize=22, textColor=NAVY)),
        Paragraph(safe(title), ParagraphStyle("doc", parent=styles["Heading1"],
                  alignment=TA_CENTER, fontSize=15)),
        Spacer(1, 12),
    ]
    for i, (heading, text) in enumerate(clauses, 1):
        story += [Paragraph(f"<b>{i}. {safe(heading)}</b>", styles["Heading3"]),
                  Paragraph(safe(text), body), Spacer(1, 6)]
    sign = Table([["_____", "_____"],
                  [safe(data["name"]), safe(data["other_party"])],
                  ["Signature / Date", "Signature / Date"],
                  ["Witness 1: _____", "Witness 2: _____"]],
                 colWidths=[85*mm, 85*mm])
    sign.setStyle(TableStyle([("TOPPADDING", (0, 0), (-1, -1), 8),
                              ("FONTSIZE", (0, 0), (-1, -1), 10)]))
    story += [Spacer(1, 24), sign, Spacer(1, 18),
              Paragraph(f"Generated on: {date.today():%d-%m-%Y}", body),
              Spacer(1, 8), Paragraph(DISCLAIMER, body)]
    doc.build(story)
    buf.seek(0)
    return buf
```

14.5 templates/index.html

```
<!doctype html>
<html lang="en"><head><metacharset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>LegalEase</title>
<style>
body{font-family:Arial,sans-serif;background:#f3f6fb;margin:0}
header{background:#17365D;color:#fff;padding:16px 40px;font-weight:bold}
main{max-width:640px;margin:30px auto;background:#fff;padding:24px;border-radius:12px}
label{display:block;margin-top:12px;font-weight:bold;font-size:14px}
input,select{width:100%;padding:8px;margin-top:4px;box-sizing:border-box}
button{margin-top:18px;background:#17365D;color:#fff;border:0;padding:10px 22px;border-radius:6px}
.err{background:#fde8e8;color:#9b1c1c;padding:10px;border-radius:6px;margin-bottom:10px}
small{color:#666}
</style></head><body>
<header>LegalEase &ndash; AI-Powered Legal Document Generator</header>
<main>
<h2>Generate a Legal Document</h2>
{% for e in errors %}<div class="err">{{ e }}</div>{% endfor %}
<form method="post" action="/generate">
<label>Document Type
<select name="document_type">
{% for k, v in document_types.items() %}
<option value="{{k}}" {{ 'selected' if form.get('document_type')==k }}>{{v}}</option>
{% endfor %}</select></label>
<label>Your Name <input name="name" value="{{ form.get('name','') }}"></label>
<label>Other Party <input name="other_party" value="{{ form.get('other_party','') }}"></label>
<label>Purpose / Property / Role <input name="purpose" value="{{ form.get('purpose','') }}"></label>
<label>Duration <input name="duration" value="{{ form.get('duration','') }}" placeholder="12 months"></label>
<label>Amount (Rs.) <input name="amount" value="{{ form.get('amount','') }}"></label>
<label>Security Deposit (Rs.) <input name="deposit" value="{{ form.get('deposit','') }}"></label>
<label>Notice Period <input name="notice" value="{{ form.get('notice','') }}" placeholder="30 days"></label>
<label>Place <input name="place" value="{{ form.get('place','') }}"></label>
<button type="submit">Generate Draft</button><br>
<small>Academic prototype only &mdash; not legal advice.</small>
</form></main></body></html>
```

14.6 templates/result.html

```
<!doctype html>
<html lang="en"><head><metacharset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>{{ title }}</title>
<style>
body{font-family:Arial,sans-serif;background:#f3f6fb;margin:0}
header{background:#17365D;color:#fff;padding:16px      40px;font-weight:bold}
h2{max-width:640px;margin:30px    auto;background:#fff;padding:24px;border-radius:12px}
ul{padding:0;border-bottom:2px        solid
button{background:#17365D;color:#fff;border:0;padding:10px 22px;border-radius:6px} </style>
</head><body>
<header>LegalEase &ndash; Generated Draft</header>
<main><h2>{{ title }}</h2>
{% for heading, text in clauses %}
<p><b>Clause {{ loop.index }}: {{ heading }}</b><br>{{ text }}</p>
{% endfor %}
<form method="post" action="/download" style="display:inline">
{% for k, v in data.items() %}<input type="hidden" name="{{k}}" value="{{v}}">{% endfor %}
<button type="submit">Download PDF</button></form>
<a href="/" style="margin-left:16px">Create another</a>
</main></body></html>
```

14.7 ai_layer.py (optional)

```
"""OPTIONAL AI layer (not used by default). Needs: pip install anthropic; ANTHROPIC_API_KEY set.
Improves clause wording only; the deterministic template is always the fallback."""
import os


def refine_clause(text, model="claude-sonnet-5-5"):
    if not os.environ.get("ANTHROPIC_API_KEY"):
        return text                      # no key -> use template text unchanged
    try:
        import anthropic
        client = anthropic.Anthropic()
        msg = client.messages.create(
            model=model, max_tokens=300,
            system="Rewrite the clause in plain formal legal English. Do not add, "
                   "remove or change names, amounts, dates or obligations. Return only the clause.",
            messages=[{"role": "user", "content": text}])
        out = msg.content[0].text.strip()
        return out if out else text
    except Exception:
        return text                      # any failure -> safe fallback
```

15. TESTING AND RESULTS

Automated tests were run with Python's `unittest` (command: `python -m unittest -v tests.test_app`). Result: **Ran 10 tests – OK.**

ID	Test case	Input	Expected	Result
T1	Home page loads	GET /	HTTP 200	Pass
T2	Valid rental preview	Valid form	200, "RENTAL AGREEMENT" shown	Pass
T3	Blank fields	Only document type	Defaults applied, no errors	Pass
T4	Invalid amount	amount = abc	HTTP 400	Pass
T5	Negative amount	-500	ValueError raised	Pass
T6	Same party names	name = other party (case-insensitive)	Error returned	Pass
T7	Unknown type PDF	document_type = x	HTTP 400	Pass
T8	for all 4 types	rental, nda, employment, leave	HTTP 200, file starts with %PDF	Pass
T9	Special characters	Name "A & B <Ltd>"	PDF created without error	Pass
T10	Long text	1000-character purpose	Truncated to 200	Pass

Manual tests recommended before the viva: open the downloaded PDF in a viewer, check the mobile-browser layout, and repeat all tests whenever templates or AI prompts change.

16. OUTPUT SCREENS

Thei nput screen shows the Document Type dropdown, party names, purpose, duration, amount, deposit, notice and place fields with a Generate Draft button and the disclaimer. The preview screen lists each numbered clause with its heading (for example Parties, Premises, Term, Rent, Security Deposit, Maintenance, Termination, Legal Compliance) and offers Download PDF and Create another. The PDF contains the LegalEase title, the clauses, a two-party signature block with two witness lines, the generation date and the disclaimer. (Insert your own screenshots of the running application here.)

17. ADVANTAGES AND LIMITATIONS

Advantages	Limitations
Simple guided interface; fast drafts	Predefined templates only; not tailored to every situation
Server-side validation and safe PDF text	No jurisdiction-specific legal validation or legal research
Signature and witness blocks included	Does not guarantee legal validity or completeness
Runs locally without an AI API	Optional AI layer needs hallucination and privacy testing
Modular, tested code; easy to demonstrate	No login, storage or history in this version

18. FUTURE ENHANCEMENTS

- Integrateth e Allayer with output checks(names, amounts and dates must be unchanged).
- Multilingual drafting (Tamil, Hindi, English) and jurisdiction/state selection.
- User login, encrypted storage, document versioning and SQLite/PostgreSQL history.
- E-signature integration, stamp-duty calculator and legally reviewed template library.
- Automated consistency checks and clause recommendations.

19. VIVA QUESTIONS AND ANSWERS

Question	Answer
Where is the AI?	The current version is template-based. ai_layer.py is an optional generative layer; the design lets AI be added without changing validation or PDF code.
Deterministic vs generative?	Deterministic: same input always gives the same output (templates). Generative: a model produces varied text and may hallucinate.
How do you prevent hallucination in legal text?	Keep the template as the source of truth, ask AI to rewrite wording only, verify names/amounts are unchanged, and require human legal review.
What happens with empty input?	Blank optional fields get defaults; an invalid amount or unknown type returns HTTP 400 with a message.
Why escape text before ReportLab?	Paragraph parses XML-like markup; unescaped < or & can break the PDF or inject markup.
Why Flask?	Lightweight, easy to learn, ideal for small web projects; integrates well with Python libraries like ReportLab.
Why generate the PDF from re-validated data?	It guarantees the PDF is built from clean data and matches the preview.
Is the output legal advice?	No. It is an educational draft and must be reviewed by a qualified lawyer.

20. CONCLUSION

LegalEasedemonstrateshowaBCA student can combine web development, input validation, document automation and AI-oriented design in a practical prototype. It provides a complete, tested flow from user input to a formatted PDF and a clear path to controlled AI integration, while stressing that automated legal text must be reviewed by qualified professionals before real-world use.

21. BIBLIOGRAPHY

- •Flaskdo cumentation– https://flask.palletsprojects.com
- •ReportLab User Guide (PDF generation with Platypus) – https://docs.reportlab.com
- •Python documentation: unittest and re modules – https://docs.python.org/3
- •OWASP guidance on input validation and output encoding – https://owasp.org
- •Nolo legal-form and legal-information resources (for understanding common document structure).
- •Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.